Backend Design Overview

Overview

This service provides a backend for managing patients and their heart rate readings. The system is designed with a focus on clean architecture, separation of concerns, and scalability, while keeping the implementation lightweight and suitable for the scope of the assignment.

The application is built using NestJS and follows a modular structure, separating domain logic, infrastructure, and cross-cutting concerns.

Architecture

The system follows a layered architecture with clear responsibility boundaries:

1. Controllers (Transport Layer)

Controllers handle HTTP requests and responses. They are responsible only for routing and delegating work to services.

2. Services (Business Logic)

Services contain all core business logic, including:

- Filtering high heart rate events
- Calculating analytics (average, min, max)
- Coordinating data access

3. Repositories (Data Access Layer)

Repositories abstract access to data. In this implementation, data is stored in memory, but the repository pattern allows easy replacement with a database in the future.

4. Infrastructure Layer

Responsible for:

- Loading seed data from a JSON file

- Providing repository implementations

5. Cross-Cutting Concerns

Handled separately from business logic:

- Request tracking implemented using an interceptor
- Input validation using DTOs and class-validator

Domain Structure

The application is divided into the following modules:

- Patients Module – handles patient retrieval and request tracking
- Heart Rate Module – manages heart rate readings and high heart rate detection
- Analytics Module – calculates statistics per patient within a time range

This modular structure improves maintainability and scalability.

Data Storage

The application uses an in-memory data store initialized from a JSON file.

Advantages:

- Simple and fast setup
- No external dependencies

Trade-offs:

- Data is not persistent
- Not suitable for distributed systems

In a production environment, this layer can be replaced with a database (e.g., PostgreSQL) without affecting business logic.

API Design

The API is designed around three main resources: patients, heart rate readings, and analytics.

Endpoints

Method	Endpoint	Description
GET	`/patients`	Get all patients with pagination
GET	`/patients/:id`	Retrieve a patient by ID
GET	`/patients/:id/request-count`	Retrieve how many times the patient's data has been requested
GET	`/heart-rate/high-events`	Retrieve all heart rate readings above 100 bpm with pagination
GET	`/heart-rate/patient/:patientId`	Get all heart rate readings for a patient
GET	`/analytics/:patientId?from=&to=`	Calculate average, minimum, and maximum heart rate values within a time range

Pagination

Collection endpoints support pagination with the following query parameters:

- offset (default: 0) – Number of items to skip
- limit (default: 10, max: 100) – Maximum number of items to return

Paginated Response Format:

```
{
  "data": [...],
  "total": 42,
  "offset": 0,
  "limit": 10,
  "hasMore": true
}
```

The API is read-only, as the assignment does not require data mutation.

Request Tracking

Patient request tracking is implemented as a cross-cutting concern.

- A singleton service (`RequestCounterService`) maintains an in-memory counter of requests per patient
- A NestJS interceptor is used to:
    - Intercept incoming HTTP requests
    - Extract `patientId` from route parameters
    - Increment the corresponding counter
    - Exclude `request-count` endpoint from counting

This approach avoids duplication and keeps tracking logic separate from business logic.

In production, this mechanism can be replaced with:

- Redis for distributed counting
- Prometheus for metrics collection

Validation

Input validation is implemented using DTOs and `class-validator`.

Specifically:

- Query parameters (`from`, `to`) are validated as ISO date strings
- Pagination parameters (`offset`, `limit`) are validated as integers with range constraints

This ensures robustness and prevents invalid input from reaching business logic.

Edge Case Handling

Special attention is given to edge cases:

Scenario	Handling
No heart rate data exists in the requested time range	Returns 0 for average, min, and max with readingsCount: 0
Patient does not exist	Returns 404 Not Found
`from` date is after `to` date	Returns 400 Bad Request

Invalid date format	Returns 400 Bad Request
No readings above 100 bpm	Returns empty array with pagination metadata
Missing required query parameters	Returns 400 Bad Request

Scalability Considerations

The system is designed with future scalability in mind:

- Repository abstraction allows replacing in-memory storage with a database
- Stateless services enable horizontal scaling
- Request tracking can be externalized to distributed systems (e.g., Redis)
- Pagination prevents large data transfers

Testing Strategy

The application includes both unit tests and e2e tests:

- Unit Tests: Cover services and business logic with mocked dependencies
- E2E Tests: Verify API endpoints, HTTP status codes, and response formats

Suggested Improvements

Given more time, the following improvements could be implemented:

1. Persistent Storage: Replace in-memory storage with PostgreSQL using TypeORM or Prisma
2. Caching Layer: Add Redis caching for analytics endpoints
3. Authentication & Authorization: Implement JWT-based authentication with role-based access
4. Rate Limiting: Add rate limiting to prevent API abuse
5. Centralized Logging: Integrate structured logging with Winston or Pino
6. Metrics Collection: Export Prometheus metrics for monitoring
7. WebSocket Notifications: Real-time alerts for high heart rate events
8. Data Export: Endpoints to export analytics in CSV or PDF format
9. API Versioning: Add `/v1/` prefix for backward compatibility
10. Comprehensive Test Coverage: Increase unit and integration test coverage
11. Integrate monitoring (e.g., Prometheus)

Conclusion

This implementation focuses on clarity, maintainability, and clean architectural principles. While simplified for the scope of the assignment, it demonstrates how the system can evolve into a production-ready service with minimal structural changes.